Experiment 2

To familiarize and understand the use and functioning of System Calls used for Operating system and network programming in Linux.

Some system calls of Linux operating systems

1. ps
This command gives processes running on the system, the owners of the processes and the names of the processes.This command tells which all processes are running on the system when ps runs.
ps -ef
UID PID PPID C STIME TTY TIME CMD
root 1 0 0 13:55 ? 00:00:01 /sbin/init

2. fork
This system call is used to create a new process. When a process makes a fork system call, a new process is created which is identical to the process creating it. The process which calls fork is called the parent process and the process that is created is called the child process. The child and parent processes are identical, i.e, child gets a copy of the parent's data space, heap and stack, but have different physical address spaces.

3. exec
New programs can be run using exec system call. When a process calls exec, the process is completely replaced by the new program. The new program starts executing from its main function.A new process is not created, process id remains the same, and the current process's text, data, heap,and stack segments are replaced by the new program. exec has many flavours one of which is execv.

4. wait
When a process terminates, its parent should receive some information reagarding the process like the process id, the termination status, amount of CPU time taken etc. This is possible only if the parent process waits for the termination of the child process. This waiting is done by calling the wait system call.

6. open
This system call is used to open a file whose pathname is given as the first parameter of the function.The second parameter gives the options that tell the way in which the file can be used.
open(filepathname , O_RDWR);

This causes the file to be read or written. The function returns the file descriptor of the file.

7. read
This system call is used to read data from an open file.
read(fd, buffer, sizeof(buffer));
The above function reads sizeof(buffer) bytes into the array named buffer. If the end of file is encountered, 0 is returned, else the number of bytes read is returned.

8. write
Data is written to an open file using write function.
write(fd, buffer, sizeof(buffer));

System calls for network programming in Linux

1. Creating a socket

```
int socket (int domain, int type, int protocol);
```

This sytem call creates a socket and returns a socket descriptor. The domain parameter specifies a communication domain; this selects the protocol family which will be used for communication. These families are defined in <sys/socket.h>. In this program the AF_INET family is used. The type parameter indicates the communication semantics. SOCK_STREAM is used for tcp connection while SOCK_DGRAM is used for udp connection. The protocol parameter specifies the protocol used and is always 0. The header files used are <sys/types.h> and <sys/socket.h>.

2. **bind()**:

This system call binds a socket to a specific local address and port, which is essential for servers to listen for incoming connections.

```
bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
```

Associates a socket with a specific local address and port.

3. listen()

Prepares a socket to accept incoming connection requests (for server sockets).

```
int listen(int sockfd, int backlog);
```

This call marks the socket as passive and sets the maximum number of incoming connections that can be queued.

sockfd: The socket file descriptor returned by socket().

backlog: The maximum number of pending connections that can be queued up.

4. accept()

Accepts an incoming connection on a listening socket.

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

This call is used by server applications to accept a connection request from a client. It returns a new socket descriptor for the accepted connection.

5. connect()

Initiates a connection to a remote server.

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

This call is used by clients to initiate a connection to a remote server.

6. send()

Sends data over a socket.

ssize_t send(int sockfd, const void *buf, size_t len, int flags);

This call sends data over a socket. It can be used for both stream and datagram sockets.

7. recv()

Receives data from a socket.

ssize_t recv(int sockfd, void *buf, size_t len, int flags);

This system call is used to receive data from a socket

8. sendto()

Sends data to a specific address (used with UDP sockets).

ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr *dest_addr, socklen_t addrlen);

This system call sends data to a specific address, which is useful for connectionless protocols like UDP.

9. recvfrom()

Receives data from a specific address (used with UDP sockets).

ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_addr, socklen_t *addrlen);

This call receives data from a specific address, which is typical for connectionless protocols like UDP.

10. shutdown()

Disables further communication on a socket.

int shutdown(int sockfd, int how);

This system call is used to shut down a socket's ability to send or receive data.

11. close()

Closes a socket.

int close(int fd);

This call is used to close an open socket and release the associated resources.